KidCheck

KidCheck Edge: IoT-Based Automatic Attendance System for Kindergarten Using Cloud-Assisted Face Recognition

IoT102 — Internet of Things · Research-Based Learning Assignment

👥 **Team Members**

Ngô Viết Bảo Huy — Phan Thị Minh Ngọc

Dương Trong — Lã Trọng Huy

FPT University
Da Nang, Vietnam
2026

🎓 Supervisor: Mr. Trần Thanh Tùng

Agenda

01 **Problem Statement**
Why traditional methods fail

02 **Solution Overview**
KidCheck at a glance

03 **System Architecture**
3-tier IoT topology

04 **Hardware Components**
ESP32-CAM & peripherals

05 **AI Pipeline — Under the Hood**
YOLO-Face + ResNet deep dive

06 **Operational Sequences**
3 core sequence diagrams

07 **Experimental Results**
Accuracy & latency data

08 **Discussion & Future Work**
Limitations & roadmap

1 Problem Statement

Current attendance methods in kindergartens are inefficient and insecure.



Manual Roll-Call

- ✗ Time-consuming
- ✗ Prone to human error
- ✗ Takes 5–10 min per class



RFID / Barcode Cards

- ✗ Children lose cards frequently
- ✗ Allows buddy-checking fraud
- ✗ No identity verification



Fingerprint Scanner

- ✗ Hygiene risk (post-pandemic)
- ✗ High Failure-to-Enroll rate
- ✗ Children's prints too small

💡 **"We need a CONTACTLESS, low-cost solution with real-time feedback to parents."**

2 Solution: KidCheck

A contactless, cloud-synchronized facial recognition attendance system.



Contactless

Face recognition—children just walk through the gate



Ultra Low-Cost

Complete edge node under $15 USD



Instant Alerts

Email notification to parents within 2 seconds

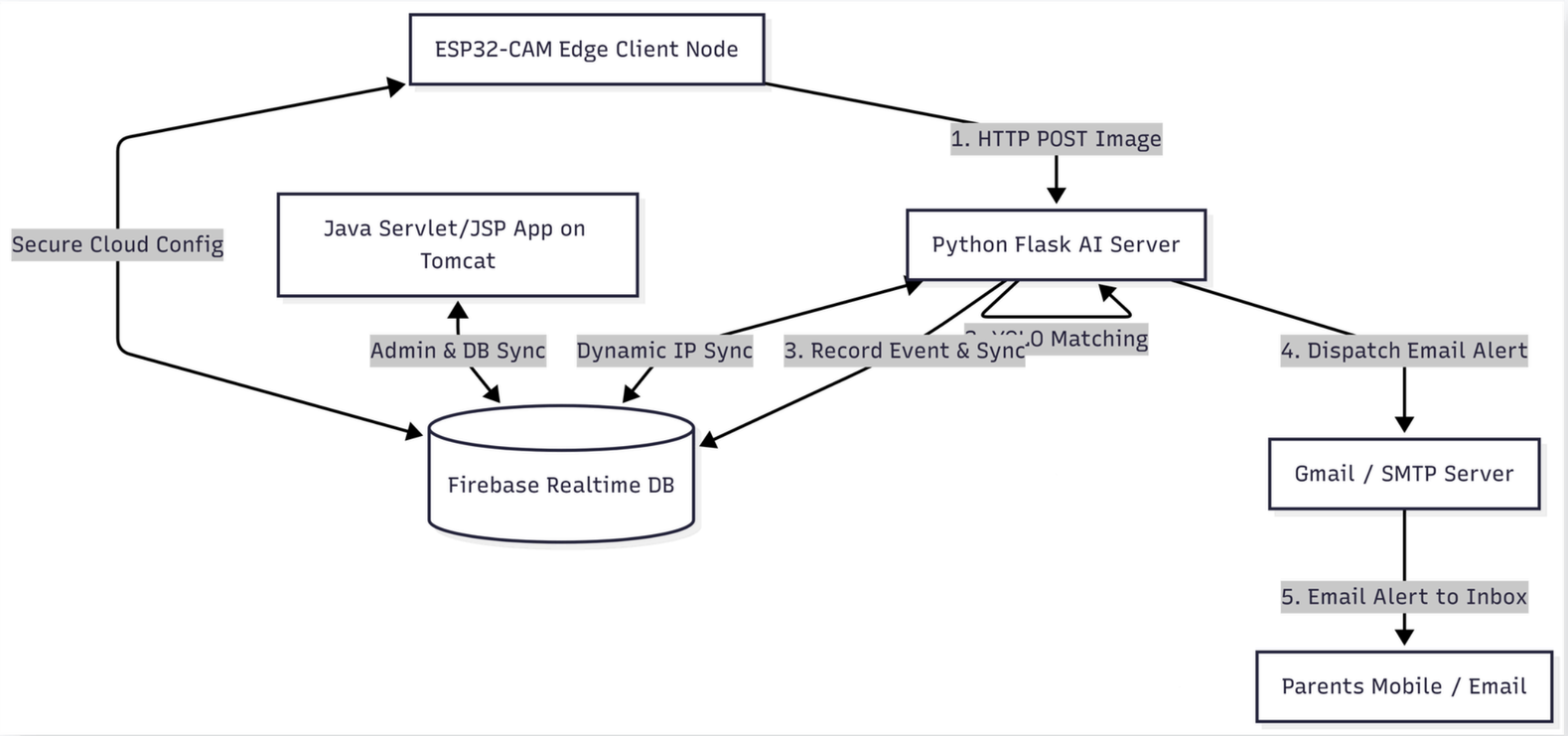


Plug & Play

Auto WiFi config (Captive Portal) + Dynamic IP sync via Firebase

🏫 Target Deployment: Kindergartens & Primary Schools — Solving child safety + attendance tracking in one system.

3 System Architecture



1 Edge Node

ESP32-CAM + PIR + LCD + Buzzer
Capture image on motion detect

2 Edge Gateway

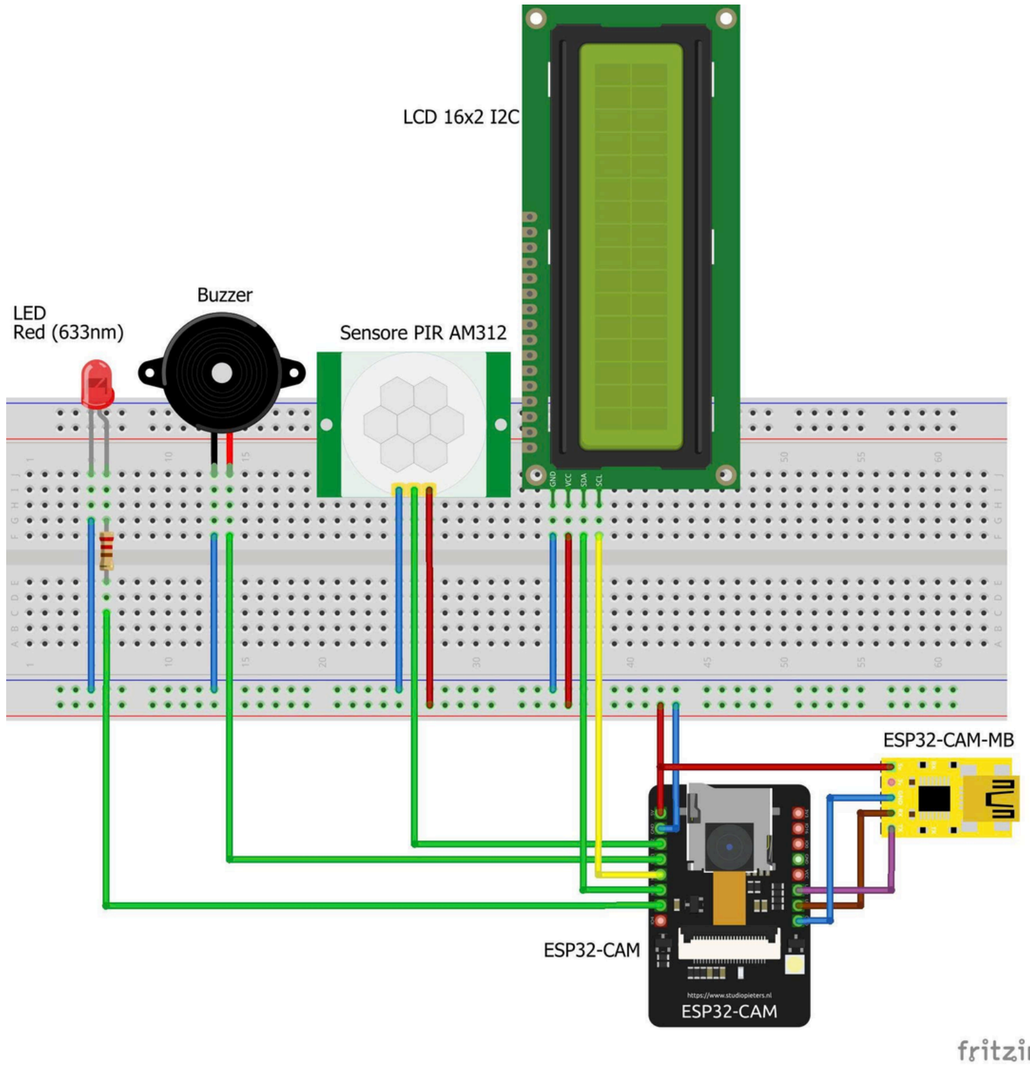
Python Flask AI Server
YOLO face detection + ResNet matching

3 Cloud & Management

Firebase RTDB + Java Servlet/Tomcat
Dashboard + Gmail

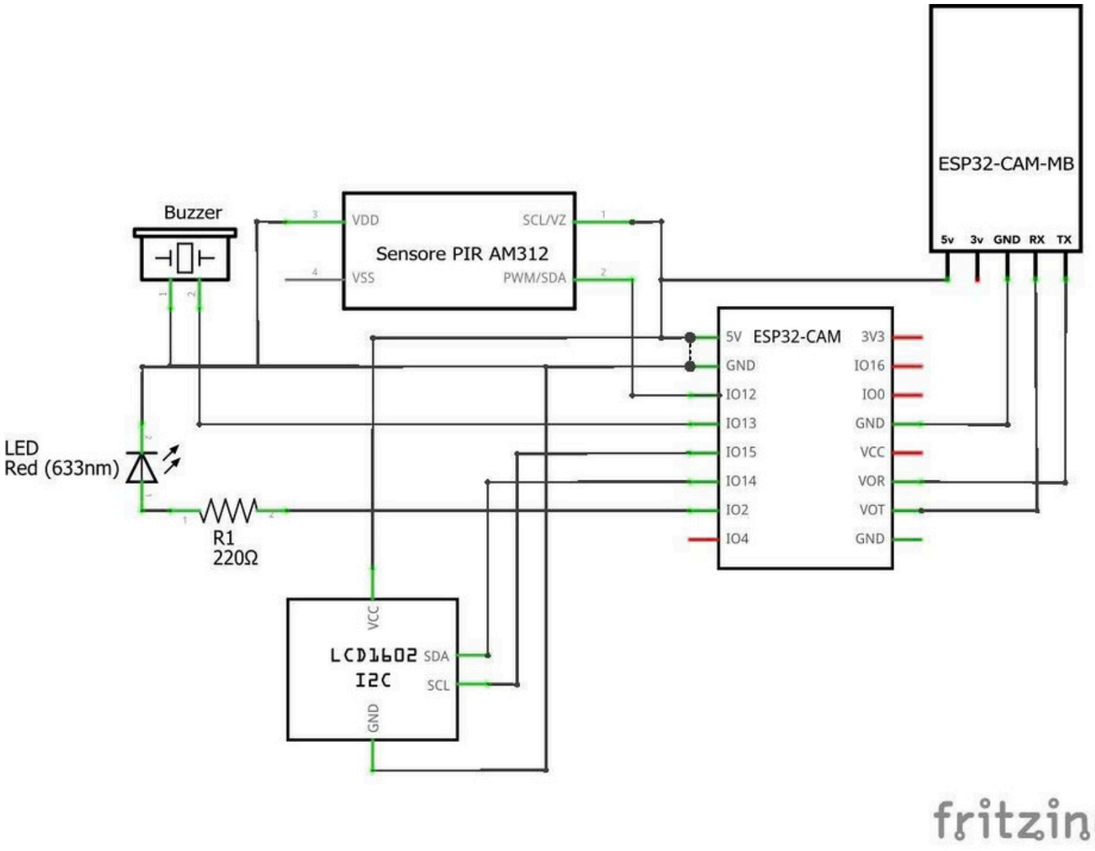
4 Hardware Components

Component	GPIO Pin	Function
ESP32-CAM (AI-Thinker)	—	Microcontroller + OV2640 Camera
ESP32-CAM-MB	—	USB programmer + 5V power supply board
PIR Sensor (SR501)	GPIO 12	Motion detection → trigger capture
LCD 1602 I2C	GPIO 14 (SDA), GPIO 15 (SCL)	Display student name / status
Active Buzzer	GPIO 13	Audio feedback (1 long beep = OK)
Status LED	GPIO 2	Indicator for active camera capture



4 Hardware Components

Component	GPIO Pin	Function
ESP32-CAM (AI-Thinker)	—	Microcontroller + OV2640 Camera
PIR Sensor (SR501)	GPIO 12	Motion detection → trigger capture
LCD 1602 I2C	GPIO 14 (SDA), GPIO 15 (SCL)	Display student name / status
Active Buzzer	GPIO 13	Audio feedback (1 long beep = OK)
Status LED	GPIO 2	Indicator for active camera capture



5 AI Pipeline — Step 1: Face Detection (YOLO)

Single-stage YOLO network predicts bounding boxes & 5 facial landmarks in real-time.

Bounding Box CIoU Loss:

$$\mathcal{L}_{box} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v$$

Multi-Task Objective:

$$\mathcal{L}_{total} = \lambda_1 \mathcal{L}_{box} + \lambda_2 \mathcal{L}_{obj} + \lambda_3 \mathcal{L}_{kpt}$$

1. Extract multi-scale feature maps via CSPDarknet / C2f backbone & PANet neck
2. Anchor-free head predicts bounding boxes (x,y,w,h) & objectness score
3. Predict 5 facial keypoints (eyes, nose, mouth) for spatial alignment
4. Non-Maximum Suppression (NMS) filters overlaps & crops facial ROI

5 AI Pipeline — Step 2: Face Encoding (ResNet-34)



Core Formula — Euclidean Distance Matching:

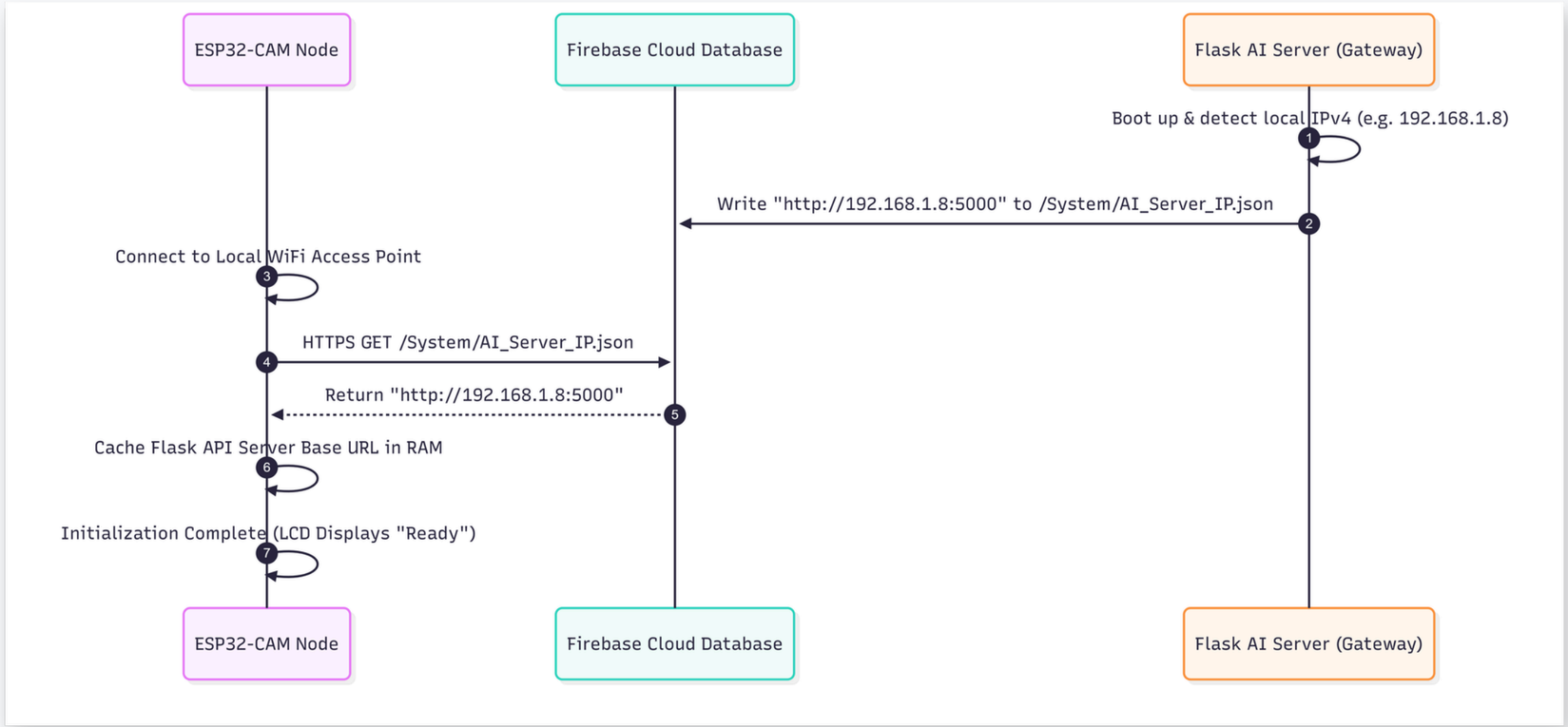
$$d(\vec{u}, \vec{v}) = \|\vec{u} - \vec{v}\|_2 = \sqrt{\left(\sum_{i=1}^{128} (u_i - v_i)^2 \right)}$$

If $d < 0.45$ → MATCH ✓ | If $d \geq 0.45$ → UNKNOWN ✗

"This is essentially the same algorithm used by Facebook/Meta for tagging friends in photos.
The face_recognition library wraps dlib's deep metric learning ResNet model under the hood."

6 Sequence 1: Auto-IP Bootstrapping

"Change your router, change your network — no code reupload needed."



→ Flask Server boots up → detects local IPv4

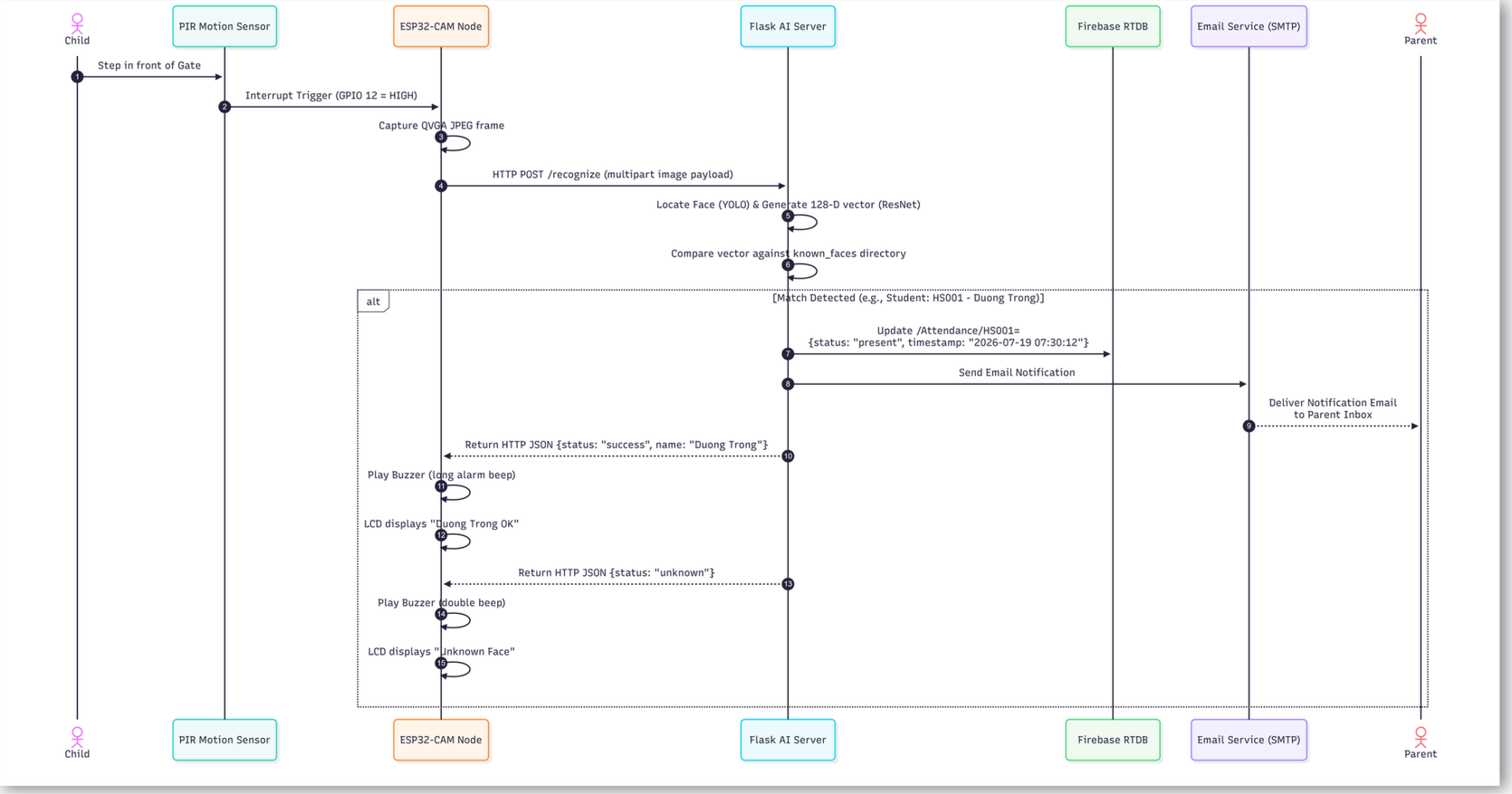
→ Writes IP to Firebase RTDB: /System/AI_Server_IP

→ ESP32-CAM boots → queries Firebase over HTTPS

→ ESP32 caches server URL in RAM

→ System fully operational — no hardcoding!

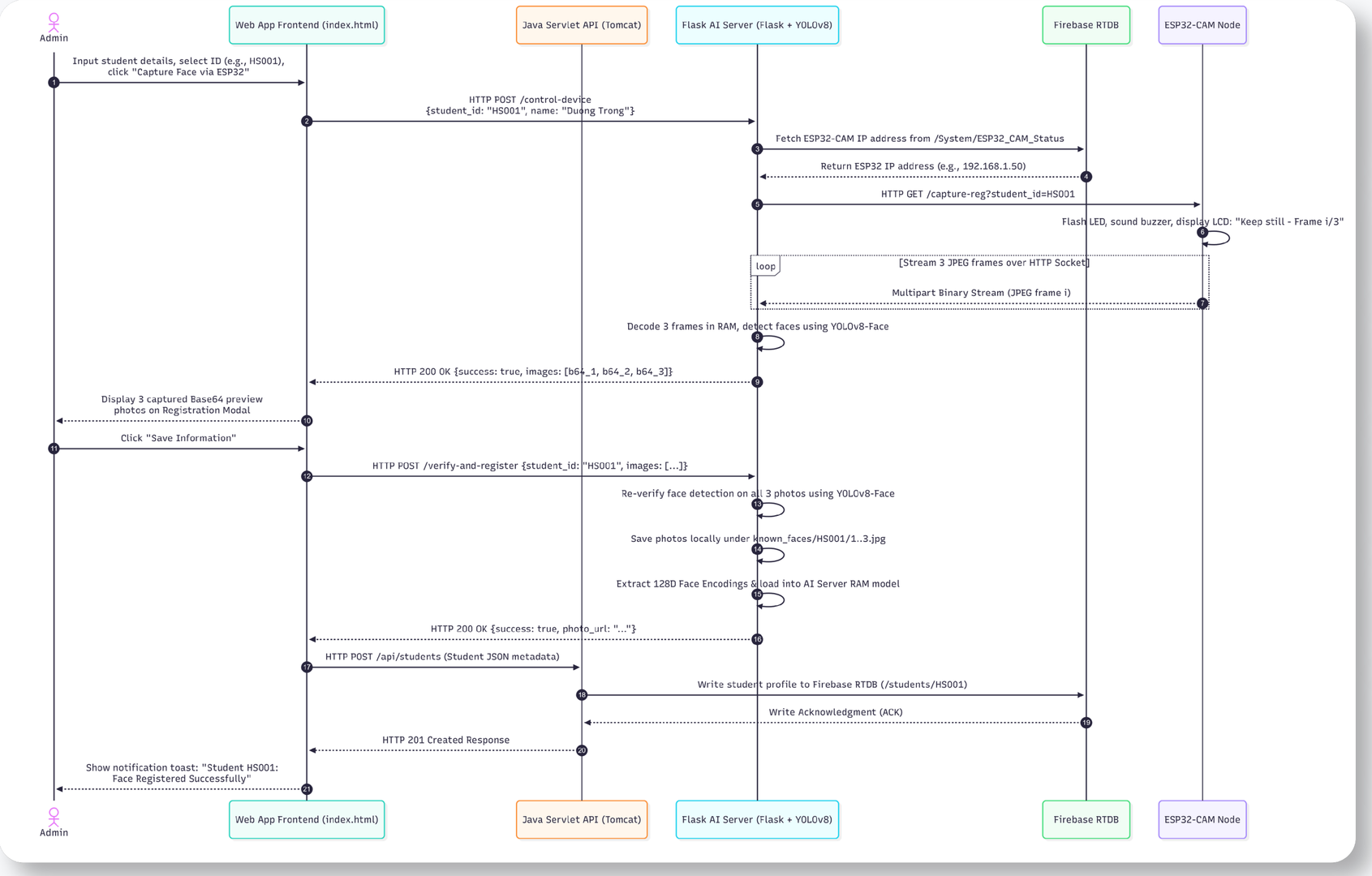
6 Sequence 2: Attendance & Parent Notification



1. **Child walks past gate**
   PIR triggers GPIO 12
2. **ESP32 captures JPEG**
   QVGA (320×240)
3. **Flask runs YOLO + ResNet**
   128-D vector matching
4. **Write result to Firebase**
   Attendance record synced
5. **Telegram webhook fires**
   Parent receives instant alert
6. **ESP32 shows result**
   LCD + Buzzer feedback

6 Sequence 3: Remote Face Enrollment

"Register a new student in 10 seconds — without touching the server."



1. Admin clicks "Capture Face" on Web Dashboard

2. Web App sends capture command directly to Flask AI Server

3. Flask fetches ESP32 IP & sends HTTP GET request to ESP32-CAM

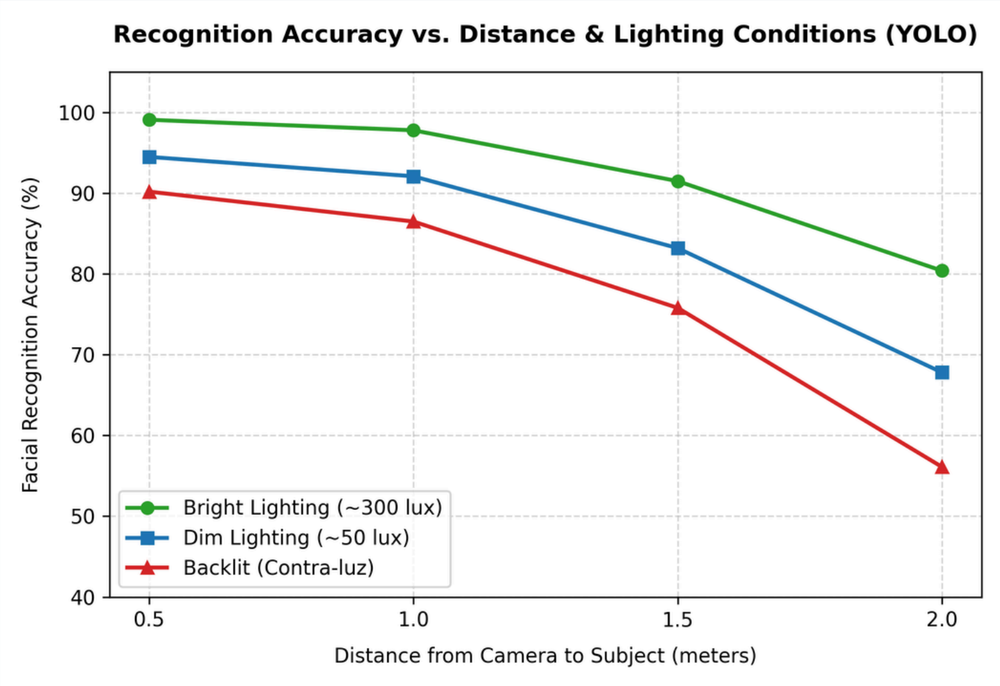
4. ESP32-CAM captures & streams 3 JPEG samples to Flask

5. YOLOv8 detects faces & returns Base64 preview to Web App

6. Admin clicks "Save" →→ YOLOv8 verifies, saves samples & generates encodings

7 Experimental Results: Recognition Accuracy

Distance (m)	Bright (~300 lux)	Dim (~50 lux)	Backlit
0.5 m	99.1%	94.5%	90.2%
1.0 m	97.8%	92.1%	86.5%
1.5 m	91.5%	83.2%	75.8%
2.0 m	80.4%	67.8%	56.1%



📌 Key Findings

✅ Optimal zone: 0.5m–1.0m, bright light → >97% accuracy

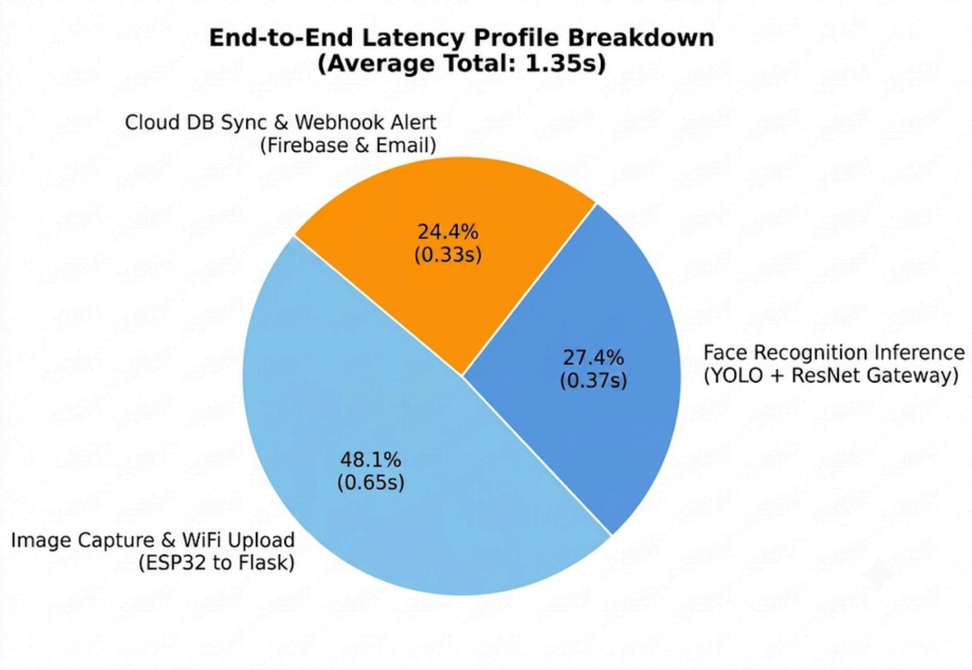
⚡YOLO multi-scale features preserve high accuracy at 1.5m and non-frontal angles

🛡️ Keypoint spatial alignment mitigates backlit silhouette distortion

💡 Recommended deployment: indoor hallway at 1.0m gate distance

7 Experimental Results: System Latency

Processing Stage	Avg Time (s)	Proportion
1. Image Capture & Upload	0.65 s	48.1%
2. Face Recognition (Flask)	0.37 s	27.4%
3. DB Sync & Email Alert	0.33 s	24.5%
Total End-to-End	1.35 s	100%



Key Insights

⚡ Total 1.35s — reduced by 25% compared to baseline HOG pipeline

🚀YOLO single-pass detection cuts AI inference latency to 0.37s (-54.8%)

📶WiFi upload is now the main component (48.1%); frame size kept ~12KB

🎯Near-instantaneous recognition at entry gates

8 Discussion & Future Work

Strengths

💰 Ultra Low Cost

< $15 USD per entry gate

🔌 Plug & Play Network

Auto IP sync via Firebase — no static IPs needed

📱 Commercial Value

Real-time parent alerts (Gmail)

🧩 Modular Design

Easy to swap AI model or add cameras

Limitations → Future Solutions

📷 Low QVGA resolution

→ Upgrade to ESP32-S3 with higher-res camera

🎭 No anti-spoofing

→ Add EAR blink detection / liveness check

🔓 HTTP unencrypted images

→ Implement HTTPS + token-based auth

⏳ Sequential processing

→ GPU acceleration / batch inference

Conclusion

KidCheck proves that low-cost IoT + open-source AI can build a smart, safe, and commercially viable attendance system for schools.

🎯

97,8%

Accuracy
at 1.0m bright light

⚡

1.35s

End-to-End
Latency

💰

< $15

Hardware Cost
per Entry Gate

Thank You! — Questions & Answers